CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 13/Unit 13: Protected Access/Friend Functions/Classes

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk
)

Unit Outline

1. Introduction
2. Protected Access
3. Friend Functions
4. Friend Classes
5. Summary

Introduction

- Recall that in C++, in general there were access modifiers `public` and `private` that we have studied earlier.
- A `public` variable can be accessed and modified by any function within or outside of the class.
- For safety, in general we keep variables as '`private`' and provide access to them using accessor (getter) and mutator (setter) functions.
- Likewise, functions, in general are declared as public, since these can be accessed by any function inside or outside of the class.
- However, private functions can also be declared.

Protected Access

- Another category of access level modifiers is **protected** access.
- Protected Access is in general, a feature of object oriented programming.
- Class members declared as **protected** can be accessed by any subclass (derived class) of that class as well.
- However, protected members cannot be accessed by classes outside of the inheritance hierarchy of the base class.

Protected Access – Example

```
#include <iostream>
using namespace std;
class Base {
protected:
    int protectedValue; // Protected member
public:
    Base() : protectedValue(0) {}
    void setValue(int val) {
        protectedValue = val;
    }
    void display() {
        cout << "Protected Value: " << protectedValue << endl;
}};
```

Protected Access – Example

```
class Derived : public Base {
public:
    void incrementValue() {
        protectedValue++; // Accessing protected member in subclass
}};
int main() {
    Derived obj;
    obj.setValue(10); // Accessible because it's public in Base
    obj.incrementValue(); // Indirectly modifies the protected
member
    obj.display(); // Outputs: Protected Value: 11
    //obj.protectedValue = 20; //Cannot access protected member
directly
    return 0;
```

Protected Access – Application/Uses

Applications/Uses

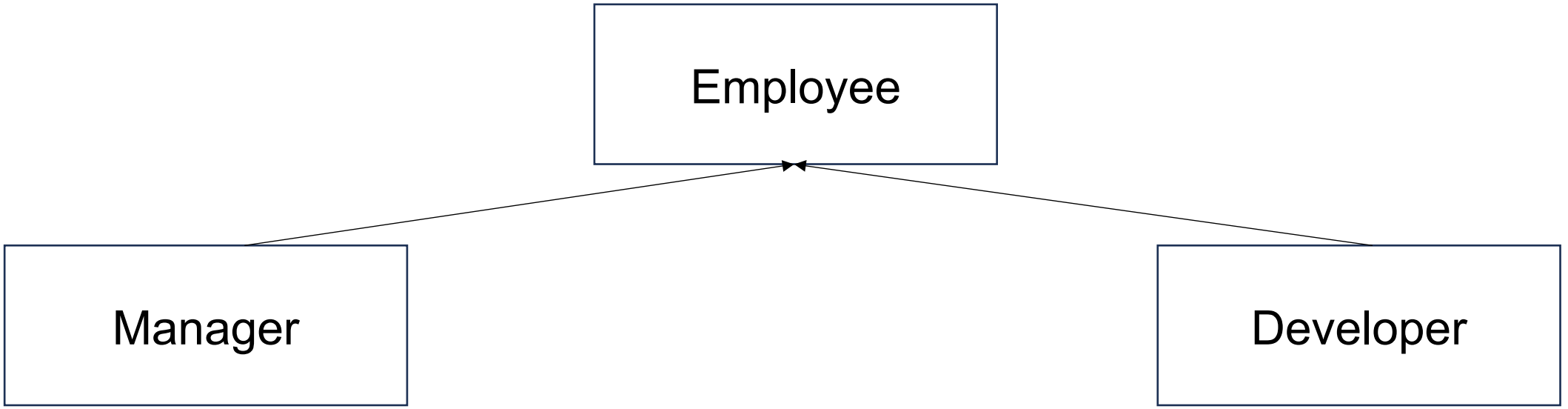
- **Protected Access** provides a middle ground between private (accessible only within the class) and public (accessible by everyone).
- It is often used to allow subclasses to use or modify inherited members without exposing those members to other parts of the program.

Benefits of Protected Access

- **Encapsulation with flexibility:** Members are hidden from the outside world but still available to subclasses.
- **Reusability:** Subclasses can reuse and extend the functionality of the base class while maintaining the integrity of its data.

Protected Access – Real World Example

- Class Demo
- UML Structure



Friend Functions

- **Friend functions** are functions that are not members of a class but are allowed access to the class's private and protected members.
- They provide a way to extend the functionality of a class without making the function a member of the class itself.
- To declare a friend function, the keyword `friend` is used inside the class definition.

Friend Functions

- The concepts of encapsulation and data hiding dictate that nonmember functions should not be able to access an object's private or protected data.
- However, there are situations where such rigid discrimination leads to considerable inconvenience.
- In this situation a **friend** function can act as a bridge between two classes.
- Example:

Friend Functions – Example – Same class

```
#include <iostream>
using namespace std;

class Box {
private:
    double length, width, height;

public:
    // Constructor
    Box(double l, double w, double h) :
    length(l), width(w), height(h) {}

    // Friend function declaration
    friend double calculateVolume(Box b);
};

// Friend function definition
double calculateVolume(Box b) {
    // Access private members directly
    return b.length * b.width *
b.height;
}

int main() {
    Box myBox(3.0, 4.0, 5.0);
    cout << "Volume of the box: " <<
calculateVolume(myBox) << endl;
    return 0;
}
```

Friend Functions – Example – Bridge 2 classes

```
#include <iostream>
using namespace std;

class beta;
//needed for frifunc declaration

class alpha
{
    private:
        int data;
    public:
        alpha() : data(3) { }
       //no-arg constructor
        friend int frifunc(alpha,
beta);
        //friend function
};
```

```
class beta
{
    private:
        int data;
    public:
        beta() : data(7) { }
       friend int frifunc(alpha,
beta);
};
int frifunc(alpha a, beta b)
//function definition
{
    return( a.data + b.data );
}
```

Friend Functions Example – Bridge 2 classes

```
//---------------------------------------------------------
int main()
{
    alpha aa;
    beta bb;
    cout << frifunc(aa, bb) << endl;
    //call the function
    return 0;
}
```

OUTPUT:

10

Some Considerations

[Encapsulation]

While friend functions can access private members, use them sparingly to maintain encapsulation principles.

Overuse can lead to tightly coupled code.

[Efficiency]

Friend functions can improve efficiency by eliminating the need for getters or setters for one-off operations.

[**Alternatives]**

If a function is closely tied to a class, consider whether it should be a member instead of a friend.

Friend Classes

- In C++, friend classes are classes that are allowed to access the private and protected members of another class.
- Declaring one class as a friend of another enables tight coupling between the two classes.
- This allows the friend class to access and modify the internals of the other class directly.
- The friendship relationship is one-way, meaning if ClassA declares ClassB as a friend, ClassB can access ClassA's private and protected members, but not vice versa unless explicitly declared.

Friend Classes – Example

```
#include <iostream>
using namespace std;

class BoxInspector;

class Box {
private:
    double length, width, height; // Private members

public:
    // Constructor
    Box(double l, double w, double h) : length(l), width(w), height(h) {}

    // Declare BoxInspector as a friend class
    friend class BoxInspector;
};
```

Friend Classes – Example

```
// Friend class
class BoxInspector {
public:
    // Method to calculate the volume of the box
    double calculateVolume(const Box& box) {
        return box.length * box.width * box.height;
       // Accessing private members of Box
    }

    // Method to display the dimensions of the box
    void displayDimensions(const Box& box) {
        cout << "Box Dimensions: "
             << box.length << " x " << box.width << " x " << box.height <<
endl;
    }
};
```

Friend Classes – Example

```
int main() {
    Box myBox(3.0, 4.0, 5.0);
    BoxInspector inspector;

    // Use the friend class to inspect the Box
    inspector.displayDimensions(myBox);
    cout << "Volume: " << inspector.calculateVolume(myBox) << endl;

    return 0;
}
```

Friend Classes – Benefits

- Why Use Friend Classes?
- Encapsulation with Collaboration:
- The BoxInspector class operates on Box's private data without exposing that data directly to the outside world.
- Cleaner Code: Eliminates the need for excessive getters and setters, which can clutter the Box class.
- Real-World Use: Useful in scenarios where one class is a natural manager of another class's data.
- This example shows how friend classes allow controlled interaction while keeping data encapsulated.

Summary

- Protected Access allows access to a class’s members